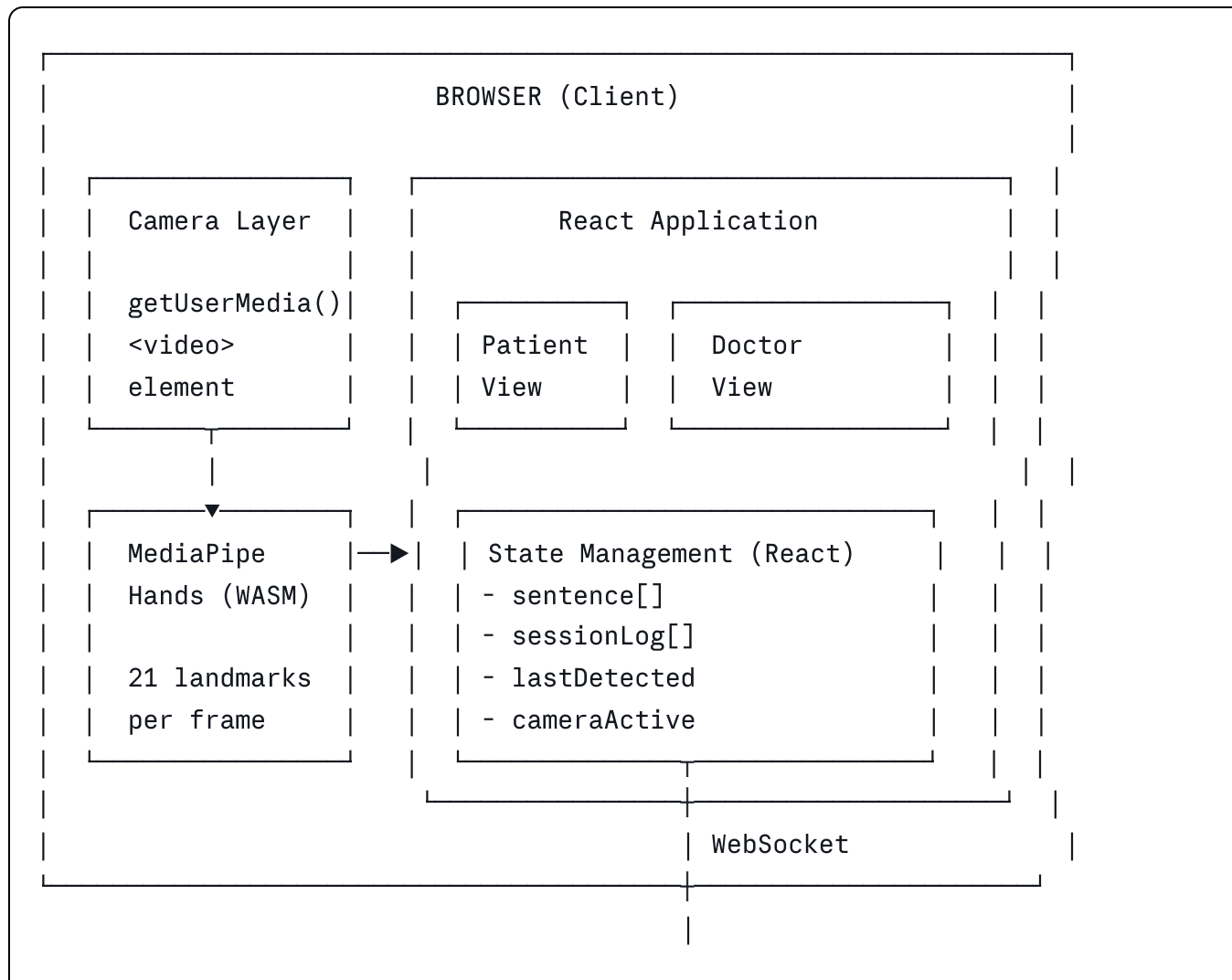


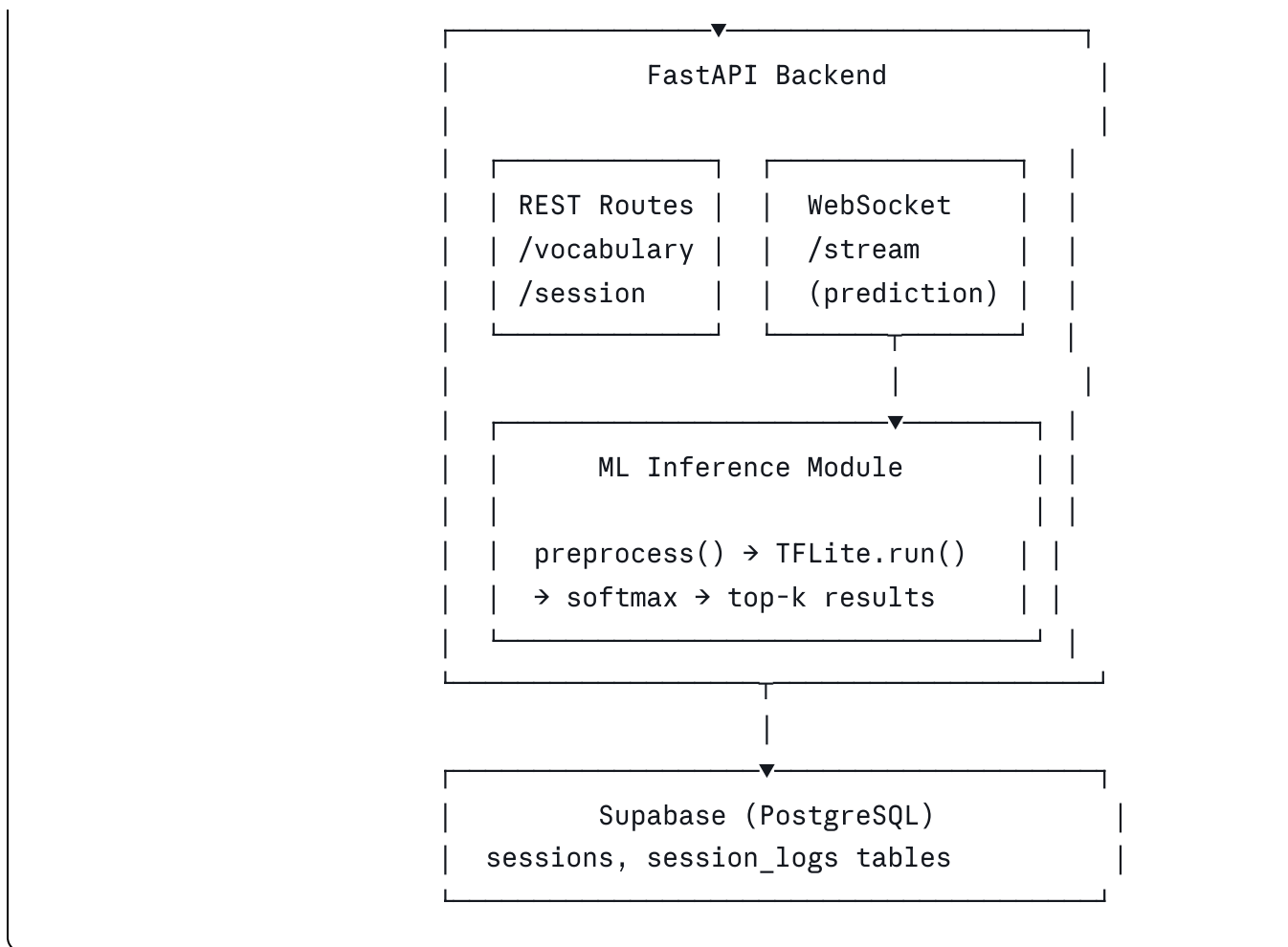
SDD.md — Software Design Document

MedSign · v1.0 · Juni 2026

1. ARSITEKTUR SISTEM

1.1 High-Level Architecture





1.2 Design Patterns

Pattern	Digunakan di	Alasan
Custom Hooks	Frontend	Memisahkan logika dari presentasi (useWebcam, useMediaPipe, useWebSocket)
Dependency Injection	Backend	ModelLoader di-inject ke route handler via FastAPI <code>Depends()</code>

Pattern	Digunakan di	Alasan
Repository Pattern	Backend	Supabase client terisolasi di <code>db/</code> folder
Observer / Event-driven	Frontend	MediaPipe <code>onResults</code> callback → trigger state update
Circuit Breaker	WebSocket	Auto-reconnect dengan exponential backoff

2. DESAIN KOMPONEN FRONTEND

2.1 Hierarki Komponen

```

App.jsx
├── Home.jsx
│   ├── RoleCard (pasien)
│   └── RoleCard (dokter)
├── PatientView.jsx
│   ├── Header
│   ├── LeftColumn
│   │   ├── CameraFeed.jsx          ← video + canvas overlay
│   │   ├── DetectionResult.jsx     ← kata + confidence bar
│   │   └── SentenceBuilder.jsx     ← akumulasi kalimat + tombol
│   └── RightColumn
│       ├── VocabularyGrid.jsx      ← tab kategori + word chips
│       └── SessionLog.jsx          ← percakapan timeline
└── DoctorView.jsx
    └── Header
  
```

```
└─ LeftColumn
  │ └─ CameraFeed.jsx      ← shared komponen
  │ └─ TranslationBig.jsx  ← tampilan besar untuk dokter
  │ └─ PatientSentence.jsx ← kalimat pasien saat ini
└─ RightColumn
  │ └─ DoctorInput.jsx     ← textarea + kirim
  │ └─ QuickPhrases.jsx    ← 10 preset frasa
  └─ SessionLog.jsx        ← shared komponen
```

2.2 Custom Hooks

javascript

```
// hooks/useWebcam.js
// Mengelola getUserMedia, stream lifecycle, cleanup
{
  videoRef,      // ref ke <video> element
  isActive,      // boolean stream status
  startCamera,    // () => Promise<void>
  stopCamera,     // () => void
  error          // string | null
}

// hooks/useMediaPipe.js
// Mengelola MediaPipe Hands instance, onResults callback
{
  canvasRef,      // ref ke <canvas> overlay
  landmarks,      // number[] (63 float) | null
  isHandDetected, // boolean
  fps             // number (detection rate)
}

// hooks/useWebSocket.js
// Mengelola koneksi WS, auto-reconnect, send/receive
{
  isConnected,    // boolean
  lastPrediction, // { prediction, confidence, top3, ms } | null
  sendFrames,     // (frames: number[][]) => void
  connectionState // 'connecting' | 'connected' | 'disconnected' | 'error'
}

// hooks/useSentenceBuilder.js
// Mengelola akumulasi kata, undo, clear
{
  sentence,      // string[]
  appendWord,    // (word: string) => void
}
```

```
removeLastWord,    // () => void  
clearSentence,     // () => void  
sentenceText      // string (joined dengan spasi)  
}
```

2.3 State Global (React Context)

javascript

```
// context/AppContext.js
const AppContext = {
  // Settings
  ttsEnabled: boolean,
  setTtsEnabled: Function,

  // Session
  sessionLog: LogEntry[],
  addLogEntry: (entry: LogEntry) => void,
  clearLog: () => void,

  // Shared state pasien ↔ dokter
  sentence: string[],
  appendWord: Function,
  removeLastWord: Function,
  clearSentence: Function,
}

// Type definitions
type LogEntry = {
  id: string, // nanoid()
  role: 'patient' | 'doctor',
  text: string,
  emoji: string | null,
  confidence: number | null,
  timestamp: string // toLocaleTimeString('id-ID')
}
```

3. DESAIN BACKEND

3.1 Struktur File

```
backend/
├─ app/
│   ├─ main.py          ← FastAPI app, startup event, CORS, router mount
│   └─ routes/
│       ├─ predict.py   ← POST /predict/gesture
│       ├─ session.py   ← POST /session/log, GET /session/{id}
│       └─ vocabulary.py ← GET /vocabulary
│   └─ ws/
│       └─ stream.py     ← WebSocket /stream
│   └─ ml/
│       ├─ model.py      ← ModelLoader singleton, predict()
│       └─ preprocess.py ← normalize_landmarks(), pad_sequence()
│   └─ db/
│       └─ supabase.py   ← client, insert_log(), get_session()
│       └─ schemas.py    ← Pydantic models
├─ models/
│   └─ medsign_v1.tflite
├─ data/
│   └─ classes.json      ← mapping index → kata
└─ requirements.txt
```

3.2 Pydantic Schemas

```
python
```

```
# schemas.py

class LandmarkFrame(BaseModel):
    values: list[float] # length = 63

class PredictRequest(BaseModel):
    frames: list[LandmarkFrame] # length = 30

class PredictionResult(BaseModel):
    prediction: str | None
    confidence: float
    top3: list[dict] # [{"word": str, "confidence": float}]
    ms: int

class LogEntry(BaseModel):
    session_id: str
    role: Literal["patient", "doctor"]
    text: str
    emoji: str | None = None
    confidence: float | None = None
    timestamp: str # ISO 8601

class VocabularyCategory(BaseModel):
    name: str
    words: list[dict] # [{"id": str, "word": str, "emoji": str, "desc": str}]
```

3.3 ML Module Design

python

```

# ml/model.py

class ModelLoader:
    """Singleton – load sekali saat startup, reuse untuk semua request."""

    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._loaded = False
        return cls._instance

    def load(self, model_path: str, classes_path: str):
        self.interpreter = tf.lite.Interpreter(model_path=model_path)
        self.interpreter.allocate_tensors()
        self.input_details = self.interpreter.get_input_details()
        self.output_details = self.interpreter.get_output_details()
        with open(classes_path) as f:
            self.classes = json.load(f) # ["SAKIT", "DEMAM", ...]
        self._loaded = True

    def predict(self, frames: np.ndarray) -> dict:
        """
        frames: shape (1, 30, 63) float32
        returns: {"prediction": str, "confidence": float, "top3": list}
        """
        start = time.perf_counter()

        self.interpreter.set_tensor(self.input_details[0]['index'], frames)
        self.interpreter.invoke()
        output = self.interpreter.get_tensor(self.output_details[0]['index'])[0]

```

```
# top-3
top3_idx = np.argsort(output)[::-1][:3]
top3 = [{"word": self.classes[i], "confidence": float(output[i])} for i in top3_idx]

ms = int((time.perf_counter() - start) * 1000)

if top3[0]["confidence"] < 0.65:
    return {"prediction": None, "confidence": top3[0]["confidence"], "time": ms}

return {"prediction": top3[0]["word"], "confidence": top3[0]["confidence"], "time": ms}
```

python

```

# ml/preprocess.py

def normalize_landmarks(landmarks_raw: list[list[float]]) -> np.ndarray:
    """
    landmarks_raw: [[x,y,z] × 21]
    returns: flat array (63,) float32, normalized
    """
    pts = np.array(landmarks_raw, dtype=np.float32) # (21, 3)

    # Shift the origin wrist
    origin = pts[0].copy()
    pts -= origin

    # Scale normalization
    distances = np.linalg.norm(pts[1:], axis=1)
    max_dist = distances.max()
    if max_dist > 0:
        pts /= max_dist

    return pts.flatten() # (63,)

def pad_sequence(frames: list[np.ndarray], target_len: int = 30) -> np.ndarray:
    """
    frames: list of (63,) arrays (variable length ≤ 30)
    returns: (1, 30, 63) float32
    """
    if len(frames) >= target_len:
        seq = np.array(frames[-target_len:], dtype=np.float32)
    else:
        padding = np.zeros((target_len - len(frames), 63), dtype=np.float32)
        seq = np.vstack([padding, np.array(frames, dtype=np.float32)])

```

```
return seq.reshape(1, target_len, 63)
```

4. DESAIN DATABASE

4.1 Schema Supabase

sql

```

-- Table: sessions
CREATE TABLE sessions (
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  facility    TEXT,          -- nama fasilitas kesehatan (opsional, pilot)
  notes      TEXT           -- catatan dokter (opsional)
);

-- Table: session_logs
CREATE TABLE session_logs (
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  session_id  UUID REFERENCES sessions(id) ON DELETE CASCADE,
  role        TEXT CHECK (role IN ('patient', 'doctor')) NOT NULL,
  text        TEXT NOT NULL,
  emoji       TEXT,
  confidence  FLOAT CHECK (confidence BETWEEN 0 AND 1),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Index untuk query session
CREATE INDEX idx_session_logs_session_id ON session_logs(session_id);
CREATE INDEX idx_session_logs_created_at ON session_logs(created_at DESC);

-- Row Level Security (privasi: setiap session hanya bisa diakses oleh pembuatn
ALTER TABLE sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE session_logs ENABLE ROW LEVEL SECURITY;

```

4.2 Data Flow — Session Log

User klik kata / dokter kirim pesan

|

▼
React: addLogEntry(entry) → update state (immediate, untuk UI)
|
▼ async (background, tidak block UI)
POST /session/log
body: { session_id, role, text, emoji, confidence, timestamp }
|
▼
Supabase: INSERT INTO session_logs
|
▼
Response: { id, status: "ok" }

5. DESAIN MODEL LSTM

5.1 Arsitektur Model

python

```

import tensorflow as tf
from tensorflow.keras import layers, Model

def build_lstm_model(n_classes: int = 40, seq_len: int = 30, n_features: int =
    inputs = tf.keras.Input(shape=(seq_len, n_features), name="landmark_sequenc

    # LSTM layers
    x = layers.LSTM(128, return_sequences=True, name="lstm_1")(inputs)
    x = layers.Dropout(0.3)(x)
    x = layers.LSTM(64, return_sequences=False, name="lstm_2")(x)
    x = layers.Dropout(0.3)(x)

    # Dense layers
    x = layers.Dense(128, activation="relu", name="dense_1")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(n_classes, activation="softmax", name="output")(x)

    model = Model(inputs, outputs, name="MedSign_LSTM_v1")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="categorical_crossentropy",
        metrics=["accuracy", tf.keras.metrics.TopK_categorical_accuracy(k=3, name
    )
    return model

```

5.2 Training Configuration

python

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_accuracy", patience=15, restore_best_weights=True
    ),
    tf.keras.callbacks.ModelCheckpoint(
        "models/best_model.h5", monitor="val_accuracy", save_best_only=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=7, min_lr=1e-6
    ),
    tf.keras.callbacks.TensorBoard(log_dir="logs/")
]

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=200,
    batch_size=32,
    callbacks=callbacks,
    class_weight=compute_class_weight("balanced", classes=np.unique(y_indices),
)
```

5.3 Export ke TFLite

python

```
# Export dengan optimasi ukuran dan kecepatan
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16] # float16 quantization
tflite_model = converter.convert()

with open("models/medsign_v1.tflite", "wb") as f:
    f.write(tflite_model)

print(f"Model size: {len(tflite_model) / 1024:.1f} KB") # Target: < 3 MB
```

6. API CONTRACT

POST `/predict/gesture`

Request:

```
Content-Type: application/json
{
  "frames": [[float × 63] × 30]
}
```

Response 200:

```
{
  "prediction": "SAKIT" | null,
  "confidence": 0.87,
  "top3": [
    {"word": "SAKIT", "confidence": 0.87},
    {"word": "NYERI DADA", "confidence": 0.09},
    {"word": "LEMAS", "confidence": 0.04}
  ],
}
```

```
"ms": 142
}
```

Response 422: Validation error (frame count tidak 30, feature count tidak 63)

Response 503: Model belum ter-load

WebSocket `/stream`

Client → Server (setiap ~1000ms):
{ "frames": [[float × 63] × 30] }

Server → Client:
{ "prediction": str|null, "confidence": float, "top3": list, "ms": int }

Koneksi ditutup oleh client saat session selesai.

Ping/pong interval: 30 detik.

GET `/vocabulary`

Response 200:

```
{
  "categories": [
    {
      "name": "Darurat",
      "words": [
        { "id": "d1", "word": "TOLONG", "emoji": "🆘", "desc": "Butuh
bantuan segera" },
        ...
      ]
    },
    ...
  ],
}
```

```
"total": 40  
}
```